Integrating Prisma into Your API

Wiring your database service into Express – controller, routes, seeding & queries

What You Built in Exercise 7

- Installed `prisma` and `@prisma/client`
- Defined `Author` and `Post` models in `schema.prisma`
- Ran `prisma migrate dev` to create the tables
- Created the `PrismaClient` singleton
- Replaced the in-memory `PostService` with a Prisma-backed one

Now you'll wire it into your controller and routes, seed the database, and explore the full Prisma toolkit.

8 – Controller

Your controller **doesn't change** – it still receives the service via the constructor:

```
import type { Request, Response } from "express";
import type { PostService } from "../services/postService.js";

export class PostController {
  constructor(private postService: PostService) {}

  async getAll(req: Request, res: Response): Promise<void> {
    const posts = await this.postService.findAll();
    res.json(posts);
  }

  async getById(req: Request, res: Response): Promise<void> {
    const id = Number(req.params.id);
    if (Number.isNaN(id)) { res.status(400).json({ error: "Invalid ID" }); return; }
    const post = await this.postService.findById(id);
    if (!post) { res.status(404).json({ error: "Post not found" }); return; }
    res.json(post);
  }

  // create, update, delete follow the same pattern ...
}
```

9 – Routes

Your router stays exactly the same – `src/routes/postRouter.ts`:

```
import { Router } from "express";
import { PostController } from "../controllers/postController.js";
import { PostService } from "../services/postService.js";

const router = Router();
const service = new PostService();
const controller = new PostController(service);

router.get("/",     (req, res) => controller.getAll(req, res));
router.get("/:id",  (req, res) => controller.getById(req, res));
router.post("/",    (req, res) => controller.create(req, res));
router.put("/:id",  (req, res) => controller.update(req, res));
router.delete("/:id", (req, res) => controller.delete(req, res));

export default router;
```

In `src/index.ts`:

```
app.use("/api/posts", postRouter);
```

10 – Seed the Database

`prisma/seed.ts`:

```
import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();

async function main() {
  const alice = await prisma.author.upsert({
    where: { email: "alice@example.com" },
    update: {},
    create: { name: "Alice", email: "alice@example.com" },
  });

  await prisma.post.createMany({
    data: [
      { title: "Introduction to Prisma", content: "A next-gen ORM for Node.js.", published: true, authorId: alice.id },
      { title: "TypeScript Tips", content: "How type safety saves time.", published: false, authorId: alice.id },
      { title: "Building REST APIs", content: "Express makes it simple.", published: true, authorId: alice.id },
    ],
    skipDuplicates: true,
  });
}

main().finally(() => prisma.$disconnect());
```

Seed Config & Run

Add to `package.json`:

```
{
  "prisma": {
    "seed": "tsx prisma/seed.ts"
  }
}
```

Run the seed:

```
npx prisma db seed
```

Reset and re-seed from scratch:

```
npx prisma migrate reset
```

> `migrate reset` drops the database, re-runs all migrations, and calls `db seed` automatically

11 – Prisma Studio

```
npx prisma studio
```

Opens at **http://localhost:5555**

- Browse all tables
- Create, edit, and delete records
- Follow relationships visually
- No SQL required

Great for **debugging** and **manual testing** during development

12 – CLI Reference

Command	What It Does
`npx prisma init`	Bootstrap schema and .env
`npx prisma migrate dev --name <n>`	Create & apply migration
`npx prisma migrate reset`	Drop DB, re-run migrations + seed
`npx prisma generate`	Regenerate the Prisma Client
`npx prisma db push`	Prototype without migration files
`npx prisma db seed`	Run the seed script
`npx prisma studio`	Open visual database browser

13 – Query Examples: Filtering & Sorting

```
// Posts by a specific author (using relation filter)
const byAuthor = await prisma.post.findMany({
  where: { author: { name: "Alice" } },
  include: { author: true },
});

// Search title
const results = await prisma.post.findMany({
  where: { title: { contains: "prisma", mode: "insensitive" } },
});

// Published posts, newest first, paginated
const page = await prisma.post.findMany({
  where: { published: true },
  orderBy: { createdAt: "desc" },
  skip: 0,
  take: 10,
});
```

13 – Query Examples: Select & Aggregate

```
// Select only specific fields (leaner response)
const summaries = await prisma.post.findMany({
  select: { id: true, title: true, published: true },
});

// Count all posts
const total = await prisma.post.count();

// Count published posts
const publishedCount = await prisma.post.count({ where: { published: true } });

// Upsert — create or update (useful in seed scripts)
const post = await prisma.post.upsert({
  where: { id: 1 },
  update: { published: true },
  create: { title: "Hello World", content: "First post!", published: false, authorId: 1 },
});
```

14 – Tips & Gotchas

Tip	Detail
Run `migrate dev` after every schema change	DB and client go out of sync otherwise
`db push` **is for prototyping only**	Never use it on a shared/production DB
Never commit `.env`	It contains your `DATABASE_URL` – always add it to `.gitignore`
`include` **is opt-in**	Relations aren't fetched unless you ask
Always `await` **Prisma calls**	Every Prisma method returns a Promise
Use `$transaction` **for multi-step writes**	Ensures all-or-nothing updates

Project Structure

```
src/
├── controllers/
│   └── postController.ts
├── routes/
│   └── postRouter.ts
├── services/
│   └── postService.ts      ← now backed by Prisma
├── prismaClient.ts         ← singleton
└── index.ts
prisma/
├── schema.prisma           ← source of truth
├── migrations/             ← migration history (commit this)
└── seed.ts                 ← seed script
```

Now it’s your turn

Exercise 8 – Wire your Prisma-backed `ExpenseService` into the full API: confirm the controller and routes need no changes, seed the database with initial data, explore it in Prisma Studio, and verify your test suite still passes.